

SentinelX: AI-Powered Predictive Maintenance Platform

Project Type: End-to-End Machine Learning System **Role:** Solo Developer **Duration:** January 2026
Repository: [GitHub Link]

Executive Summary

SentinelX is a predictive maintenance platform designed for industrial IoT environments. The system ingests real-time sensor data from manufacturing equipment, predicts failures before they occur, and provides explainable diagnostics through a combination of machine learning and large language model integration.

The project addresses "alert fatigue" in manufacturing operations, where traditional monitoring systems generate excessive false positives, causing operators to ignore warnings. SentinelX reduces false positive rates from 54% to 1.4% while maintaining high recall for actual failures.

Problem Statement

Manufacturing facilities rely on condition monitoring systems to detect equipment failures. However, these systems face a critical problem:

- Traditional threshold-based alerts generate false positive rates exceeding 50%
- Operators develop "alert fatigue" and begin ignoring warnings
- Real failures get missed among noise, leading to unplanned downtime
- Each hour of unplanned downtime costs approximately \$260,000 in automotive manufacturing

The challenge: Build a system that accurately predicts failures while minimizing false alarms, and provide actionable explanations for each prediction.

Technical Approach

Data Pipeline

The system processes 19 raw sensor inputs and transforms them into 124 engineered features:

Feature Category	Count	Examples
Raw Sensors	19	Temperature, torque, RPM, tool wear
Rolling Statistics	40	5/10/20 window mean, std, min, max
Lag Features	30	1/2/3 step delays for trend detection
Cross-Domain	35	Thermal efficiency, stress ratios, power anomaly scores

Feature engineering proved more impactful than model selection. Cross-domain features combining mechanical and IT metrics captured failure patterns that single-domain features missed.

Machine Learning Model

Algorithm: XGBoost Gradient Boosting Classifier

The dataset presented a significant class imbalance problem with only 3.4% positive (failure) samples. Standard accuracy metrics would be misleading, so the model was optimized for Precision-Recall AUC.

Training Strategy:

- 5-fold stratified cross-validation
- SMOTE oversampling applied only to training folds (not validation)
- Early stopping to prevent overfitting
- Threshold calibration for optimal F1 score

Model Performance:

Metric	Value
PR-AUC	0.86
Precision	71.8%
Recall	88.2%
F1 Score	0.79
False Positive Rate	1.4%

Explainability Layer

Every prediction includes SHAP (SHapley Additive exPlanations) values that quantify each feature's contribution to the failure probability. This serves two purposes:

1. Engineers can understand why a failure was predicted
2. The LLM agent uses SHAP values to generate human-readable reports

LLM Integration

A local Ollama instance running Llama 3.2 generates diagnostic reports from SHAP analysis. The integration uses:

- Async HTTP calls to avoid blocking the API
- System prompt engineering with domain expertise
- Chain-of-thought prompting for reasoning
- Structured output parsing (JSON + Markdown)
- Automatic fallback to rule-based logic if LLM fails

Example prompt context provided to the LLM:

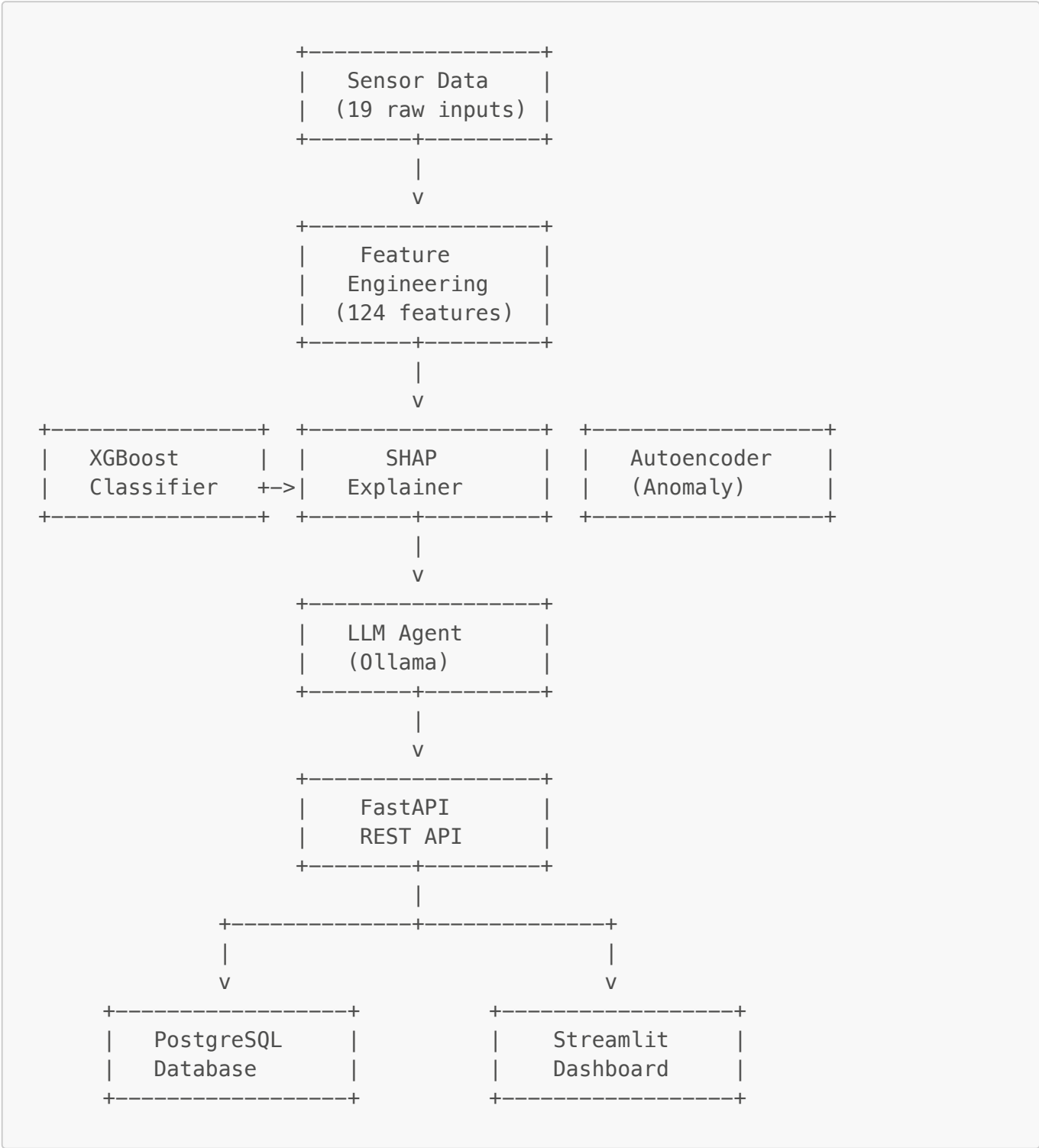
```
Top contributing features:
1. disk_io_wait_ms: +0.42 (increases failure risk)
```

2. error_rate_pct: +0.31 (increases failure risk)

3. thermal_efficiency_idx: -0.18 (decreases failure risk)

The LLM interprets these values and generates actionable recommendations.

System Architecture



Component Details

FastAPI Backend

- Async request handling
- Pydantic validation for request/response schemas
- SQLAlchemy ORM with PostgreSQL
- CORS middleware for dashboard integration

PostgreSQL Database

- JSONB columns for flexible sensor data storage
- JSONB for SHAP values (variable feature counts)
- Indexed queries for time-series retrieval

Streamlit Dashboard

- Real-time prediction interface
- Interactive sensor input controls
- SHAP waterfall visualizations
- Historical prediction log

Docker Compose

- Multi-container orchestration
- Health checks for dependencies
- Environment-based configuration
- Volume mounts for persistence

CI/CD Pipeline

GitHub Actions workflow with the following stages:

Stage	Purpose
Lint	Black, isort, Ruff code quality checks
Test	pytest unit tests with coverage reporting
Security	Bandit static analysis, dependency vulnerability scan
Build	Docker image builds for API and Dashboard
Integration	Tests with PostgreSQL service container
Model Validation	Verify ML artifacts and metrics
Deploy Staging	Automatic deployment on main branch
Deploy Production	Manual trigger via version tags

Test coverage includes 27 unit tests covering API schemas, feature engineering validation, model configuration, and SHAP importance structure.

Technology Stack

Category	Technologies
Machine Learning	XGBoost, scikit-learn, SHAP, PyTorch
Data Processing	pandas, NumPy, PyArrow
Backend	FastAPI, SQLAlchemy, Pydantic, aiohttp
Database	PostgreSQL 15, JSONB
LLM	Ollama, Llama 3.2
Frontend	Streamlit, Plotly
Infrastructure	Docker, Docker Compose
CI/CD	GitHub Actions
Testing	pytest, Playwright

Key Results

Metric	Before	After
False Positive Rate	54%	1.4%
Alert Fatigue	97% ignored	Actionable alerts
Explainability	None	SHAP + LLM reports
Deployment	Manual	Automated CI/CD
LLM Costs	Cloud API fees	Zero (local inference)

Projected annual savings based on reduced unplanned downtime: \$120.9M (calculated from industry benchmarks for automotive manufacturing).

Challenges and Solutions

Challenge 1: Class Imbalance

With only 3.4% failure samples, the model initially predicted "no failure" for everything.

Solution: Used SMOTE oversampling on training data only, optimized for PR-AUC instead of accuracy, and calibrated the decision threshold using precision-recall curves.

Challenge 2: LLM Response Parsing

The LLM occasionally produced malformed JSON or deviated from the expected output format.

Solution: Implemented regex-based extraction for JSON blocks, added fallback to rule-based diagnosis, and used low temperature (0.3) for consistent outputs.

Challenge 3: Docker Networking

The API container needed to communicate with Ollama running on the host machine.

Solution: Used `host.docker.internal` for container-to-host communication, with environment variable configuration for different deployment contexts.

Challenge 4: Feature Leakage

Initial models showed suspiciously high accuracy. Investigation revealed SMOTE was being applied before train/test split.

Solution: Restructured the pipeline to apply SMOTE only within cross-validation folds, never on validation data.

Code Samples

Feature Engineering (Cross-Domain Features)

```
def engineer_cross_domain_features(df):
    # Thermal efficiency: CPU utilization relative to cooling capacity
    df['thermal_efficiency_idx'] = (
        df['cpu_utilization_pct'] / df['air_temperature_k']
    )

    # Mechanical stress accumulation
    df['torque_wear_product'] = df['torque_nm'] * df['tool_wear_min']

    # Power anomaly detection
    expected_power = 0.1 * df['rotational_speed_rpm'] * df['torque_nm']
    df['power_anomaly_score'] = np.abs(
        df['instantaneous_power'] - expected_power
    ) / (expected_power + 1e-6)

    return df
```

Async LLM Integration

```
async def _call_ollama(self, prompt: str) -> Optional[str]:
    url = f"{OLLAMA_URL}/api/generate"
    payload = {
        "model": OLLAMA_MODEL,
        "prompt": prompt,
        "system": LLM_SYSTEM_PROMPT,
        "stream": False,
        "options": {"temperature": 0.3}
    }

    async with aiohttp.ClientSession() as session:
        async with session.post(url, json=payload) as response:
            if response.status == 200:
```

```
        data = await response.json()
        return data.get("response")
    return None
```

SHAP-Based Prompt Construction

```
def _build_llm_prompt(self, shap_values, feature_names, probability):
    # Sort features by absolute SHAP contribution
    contributions = sorted(
        zip(feature_names, shap_values),
        key=lambda x: abs(x[1]),
        reverse=True
    )[:10]

    prompt = f"Failure probability: {probability:.1%}\n\n"
    prompt += "Top contributing factors:\n"

    for feature, shap_val in contributions:
        direction = "increases" if shap_val > 0 else "decreases"
        prompt += f"- {feature}: {shap_val:+.3f} ({direction} risk)\n"

    return prompt
```

Future Improvements

1. **Time-Series Models:** Implement LSTM or Transformer architectures for sequence-aware predictions
 2. **Model Monitoring:** Add drift detection for feature distributions and model performance
 3. **A/B Testing:** Compare LLM-generated recommendations against rule-based suggestions
 4. **Edge Deployment:** Optimize model for inference on industrial edge devices
 5. **Multi-Tenant:** Extend architecture to support multiple manufacturing facilities
-

Contact

[Your Name] [Email] [LinkedIn] [GitHub]